Physics IDE — User Guide

> **Version:** 1.0 · **Date:** March 2026
> **Audience:** Students, Educators, and Developers

Table of Contents

---

1. What is Physics IDE?

Physics IDE is a browser-based 3D physics simulation environment. It lets you create, run, and debug physically accurate 3D simulations without installing any software. Everything runs in the browser.

Two ways to build

MODE	BEST FOR	HOW IT WORKS
Block Editor	Visual learners, beginners, no-code	Drag colour-coded blocks together; code is generated automatically
Code Editor	Python learners, advanced users	Write VPython directly with syntax highlighting and auto-complete

Both modes drive the **same 3D simulation engine** (GlowScript/VPython 3.2). You can switch between them at any time — your work is preserved.

Physics engine highlights

- **Real 3D WebGL rendering** — orbit, pan, and zoom with the mouse.
- **VPython objects** — spheres, boxes, cylinders, arrows, helices, labels, trails, lights.
- **Physics primitives** — gravity, velocity, acceleration, force, spring mechanics, momentum, energy.
- **3D Math** — vectors, cross/dot products, magnitude, rotation, clamp.
- **Simulation control** — time step, forever loop, rate limiter, conditional logic.

---

2. Getting Started

Step 1 — Open the IDE

Open Physics IDE in a modern browser (Chrome, Firefox, Edge, or Safari 15+). No login is required. You will see the **Start Menu**.

Step 2 — Choose a starting point

The Start Menu offers:

- **Precoded examples** (Blocks or Code): Ready-to-run physics simulations.
- **Blank project** (Blocks or Code): An empty workspace — start from scratch.

Step 3 — Click Run

Once a template or blank project loads, press the green **Run** button in the toolbar. The 3D simulation viewport on the right will come to life.

Step 4 — Modify and experiment

Change a physics constant, add a block, or edit the code. Click **Run** again to see the effect. Physics IDE **auto-saves your workspace** every 2 seconds — you never need to manually save.

---

3. The Start Menu

The Start Menu appears when Physics IDE first opens (or when you click the **Menu** button in the toolbar to return to it).

Template cards

Each card shows:

- A title and short description of the simulation.
- A badge indicating whether it opens in **Blocks** or **Code** mode.
- A **difficulty level** (Beginner / Intermediate / Advanced).

Filter bar

Click **Blocks**, **Code**, or **All** to filter the available templates.

Included templates

TEMPLATE	MODE	PHYSICS	LEVEL
Projectile Motion	Code	Ballistics, air drag, bounce, ground collision	Intermediate
Spring-Mass Oscillator	Code	Hooke's law, damping, kinetic/potential energy	Intermediate
Electric Field (3 charges)	Code	Coulomb's law, field vectors, potential	Advanced
Blank (Blocks)	Blocks	Start from scratch	Beginner
Blank (Code)	Code	Start from scratch	Beginner

---

4. The Toolbar

The toolbar runs across the top of the screen. It contains all primary actions.

Navigation

BUTTON	ACTION
Menu	Return to the Start Menu (confirms if you have unsaved changes)
Help	Open the in-app documentation page

Simulation controls

BUTTON	SHORTCUT	ACTION
Run	`Ctrl+Enter`	Compile and run the VPython simulation
Stop	—	Stop the currently running simulation
Reset	—	Stop the simulation and reset blocks/code to the last saved state

Workspace controls

BUTTON	ACTION	AVAILABLE IN
Clear	Remove all blocks from the workspace	Blocks mode only
Mode Toggle	Switch between Block Editor and Code Editor	Both

Export

BUTTON	FORMAT	CONTENT
Export .py	Python source file	The generated or written VPython code
Export .xml	Blockly workspace XML	The full block workspace (can be re-imported)
Export Blocks PDF	PDF	Screenshot of the block diagram
Export Code PDF	PDF	Syntax-highlighted code as PDF
Screenshot	PNG image	Current state of the 3D viewport
Copy Code	Clipboard	The Python code, copied to clipboard

View controls

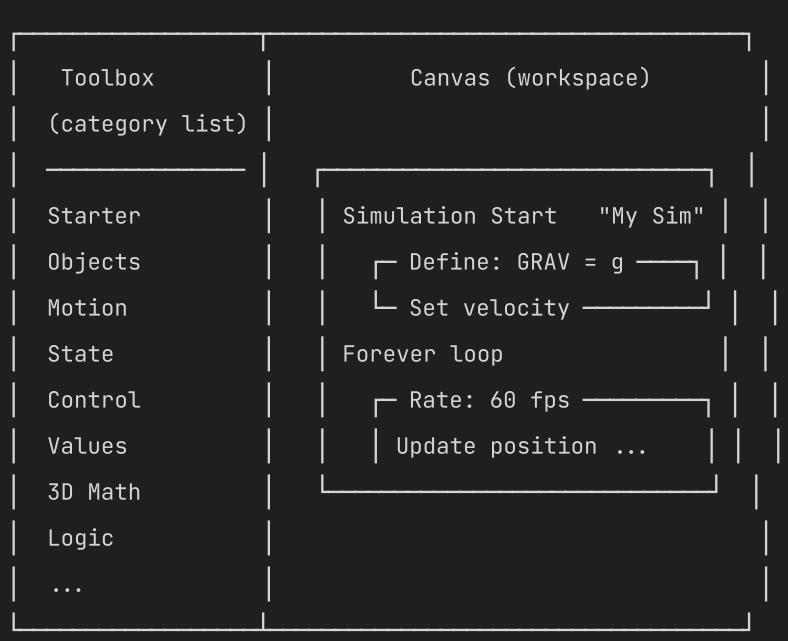
BUTTON	ACTION
Viewport	Show/hide the 3D simulation panel
Trace	Show/hide the live variable trace table
Debug	Enter Debug Mode
Beginner Mode	Toggle simplified block toolbox (Blocks mode only)
Zoom slider	Adjusts viewport or editor zoom level
Theme	Toggle dark ↔ light theme

---

5. Block Editor

The Block Editor uses **Google Blockly v11** — the same engine used in Scratch and MIT App Inventor.

Layout



Using the toolbox

1. Click a category name to expand it.
2. Drag any block onto the canvas.

3. Connect blocks by snapping them together — compatible connectors glow when close.
4. Fill in field values (numbers, text, colours) by clicking directly on them.

Block search

Use the **search bar** at the top of the toolbox to find blocks by name or keyword. Results show the block type, category, and matching keywords.

Beginner Mode

Click **Beginner Mode** in the toolbar to show only essential blocks:

- **Starter** (simulation structure)
- **Objects** (sphere, box)
- **Motion** (velocity, position update, gravity)
- **Control** (forever loop, rate, if)
- **Values** (vectors, numbers)
- **Variables**

Context menu (right-click)

Right-click any block to:

- **Duplicate** the block and its children.
- **Delete** the block.
- **Add comment** (yellow comment bubble).
- **Collapse block** to save canvas space.
- **Help** — link to Blockly documentation.

Undo / Redo

ACTION	SHORTCUT
Undo	`Ctrl+Z`
Redo	`Ctrl+Y` or `Ctrl+Shift+Z`

Code mirror panel

Switch to the **Code Editor** any time to see what Python code your blocks have generated. The code updates live as you add or change blocks.

---

6. Code Editor

The Code Editor uses **Monaco Editor** — the same engine as Visual Studio Code.

Features

- **Syntax highlighting**: VPython keywords, built-ins, strings, numbers, and comments are colour-coded.
- **Auto-indent**: Pressing Enter after a `:` (e.g., `while True:`) auto-indents the next line.
- **Line numbers**: Visible on the left margin.
- **Minimap**: Right-side overview of the whole file.
- **Find & Replace**: `Ctrl+H`

VPython auto-complete

The editor provides basic IntelliSense for VPython keywords (`sphere`, `box`, `vector`, `color`, `rate`, `mag`, `norm`, etc.).

Starting code structure

Every VPython program must begin with the GlowScript header:

```
GlowScript 3.2 VPython
```

Then set up your scene and objects:

```
scene.title = "My Simulation"
scene.background = vector(0.1, 0.1, 0.15)

# Create a sphere
ball = sphere(pos=vector(0, 5, 0), radius=1, color=color.red)

# Set physics parameters
g = vector(0, -9.81, 0)
dt = 0.01
ball.velocity = vector(2, 0, 0)

# Main loop
while True:
    rate(100)
    ball.velocity = ball.velocity + g * dt
    ball.pos = ball.pos + ball.velocity * dt
    if ball.pos.y < ball.radius:
        ball.pos.y = ball.radius
        ball.velocity.y = -ball.velocity.y * 0.8
```

Switching from Blocks to Code

When you switch from Block mode to Code mode, the generated Python code is loaded into the editor. Changes made in Code mode **do not sync back to blocks** (code → blocks conversion is not supported in this release).

---

7. 3D Simulation Viewport

The right panel is the **3D Simulation Viewport** — a sandboxed window running GlowScript/VPython.

Camera controls

ACTION	RESULT
Left-click + drag	Rotate the camera around the scene
Right-click + drag	Pan the camera
Scroll wheel	Zoom in / zoom out
Middle-click + drag	Pan (alternative)

During a simulation

- Objects in the scene move and animate in real time.

- `print()` statements and `label()` objects appear overlaid on the 3D scene.
- The `scene.caption` text area below the viewport shows formatted messages.

Viewport visibility

Click **Viewport** in the toolbar to hide the 3D panel and give the editor more space. The simulation continues running in the background.

---

8. Debug Mode

Debug Mode is a **full-screen overlay** that lets you step through your simulation line by line, set breakpoints, and watch variable values change in real time.

Entering Debug Mode

Click **Debug** in the toolbar. The IDE will:

1. Show the Python code alongside the Blockly block diagram (read-only).
2. Provide playback controls at the top.

Debug controls

CONTROL	ACTION
Pause (or `Space`)	Pause execution at the next instrumented line
Resume	Continue running after pause
Step (or `F10`)	Execute one line and pause again
Exit Debug	Return to normal editing mode

Breakpoints

- **In code view**: Click any line number to toggle a red breakpoint dot.
- **In block view**: Click any block to toggle a red dot on that block.

When the simulation hits a breakpoint, execution pauses and that line/block is highlighted in **yellow**.

Recording

While in Debug Mode, click **Record** to capture variable snapshots at each step. Recordings can be exported as CSV for analysis.

Execution highlight

The currently executing block is highlighted with a **yellow glow** in the block diagram. The corresponding code line is highlighted in the code panel simultaneously.

---

9. Live Trace Table

The Trace Table monitors the values of tracked variables **in real time** as a simulation runs.

Opening the Trace Table

Click **Trace** in the toolbar to show the trace panel below the viewport.

What gets traced

Any variable tagged with `__trace__` in the Python code is automatically monitored. When you build with blocks, tracing is configured via the **Telemetry** blocks in the State category.

Trace table columns

COLUMN	DESCRIPTION
Variable	Name of the tracked variable
Value	Current value (updates live)
Δ (Delta)	Change since last update
Min	Minimum value seen this session
Max	Maximum value seen this session
Sparkline	Mini graph of the last 60 data points

Controls

BUTTON	ACTION
Pin	Keep a row at the top regardless of sort order
Search	Filter variables by name
Export CSV	Download all recorded trace data as a CSV file
Clear	Reset all trace history

10. Exporting Work

Physics IDE supports multiple export formats to let you share, submit, or archive your simulations.

Python source (`.py`)

Click **Export .py** to download the current VPython code as a `.py` file. This file can be:

- Run locally using a full Python + VPython installation.
- Submitted to an instructor as coursework.
- Shared with other students.

Blockly workspace (`.xml`)

Click **Export .xml** to download the full Blockly workspace as an XML file. This preserves every block, connection, and field value. Can be re-imported exactly.

Block diagram PDF

Click **Export Blocks PDF** to generate a PDF containing a screenshot of the current block canvas. Useful for:

- Homework submission showing block structure.
- Printing for class discussion.

Code PDF

Click **Export Code PDF** to generate a PDF of the Python code with full syntax highlighting. The code is rendered with colour-coded tokens (keywords, strings, comments, numbers).

Screenshot

Click **Screenshot** to capture the current 3D viewport as a PNG image. The image downloads immediately.

Copy to clipboard

Click **Copy Code** to copy the entire Python source to the clipboard. Useful for pasting into an online submission form or another editor.

---

11. Importing Work

Import `.xml` (Blockly workspace)

Click **Import** in the toolbar and select a `.xml` file. The block workspace is restored exactly as it was when exported.

Import `.py` (Python code)

Select a `.py` file to load its contents into the Code Editor. The mode automatically switches to Code mode.

> **Note:** Importing a `.py` file does not recreate blocks — it loads the raw Python source into the Code Editor.

---

12. Themes & Display

Dark / Light theme

Click the **theme icon** (sun/moon) in the toolbar to toggle between dark and light themes. The preference is saved to `localStorage` and persists across sessions.

Dark theme (default)

VS Code-inspired dark theme with deep navy backgrounds, blue accents, and high-contrast code highlighting.

Light theme

Clean light background with muted blue accents — suitable for printed documentation and bright environments.

Blockly theme

The block editor automatically adapts its colour palette when you toggle the theme — dark mode uses darker backgrounds for the canvas and panels, light mode uses white/grey.

---

13. Keyboard Shortcuts

SHORTCUT	ACTION
`Ctrl+Enter`	Run simulation
`Ctrl+Z`	Undo (in Blockly or Monaco)
`Ctrl+Y`	Redo
`Ctrl+H`	Find & Replace (Monaco only)
`Ctrl+/`	Toggle comment (Monaco only)
`Space`	Pause / resume (in Debug Mode)
`F10`	Step (in Debug Mode)
`Escape`	Close Help / Exit Debug Mode

---

14. Block Reference

Starter Category

BLOCK	DESCRIPTION
Simulation Start	Marks the beginning of a simulation; contains setup blocks
Simulation End	Marks the end; prints a completion message
Quick Sphere (preset)	Creates a sphere with position, radius, and colour in one block
Quick Box (preset)	Creates a box/floor/wall with size, position, and colour
Set Gravity	Sets the global gravity vector
Time Step	Defines `dt` — the simulation time increment per loop iteration
Forever Loop	Main simulation loop (wraps VPython's `while True: rate(...)` pattern)
Update Position	Applies `pos += velocity * dt` to a named object

Objects Category

BLOCK	OBJECT TYPE	NOTES
Sphere	`sphere()`	Position, radius, colour, opacity, emissive
Sphere + trail	`sphere()` with `make_trail=True`	Trail radius, trail colour, retain count
Glowing sphere	`sphere()` with emissive	Star / light glow effect
Box	`box()`	Size (`width × height × depth`), colour
Box (transparent)	`box()` with opacity	Opacity 0–1
Cylinder	`cylinder()`	Start position, axis vector, radius, colour
Arrow	`arrow()`	Position, axis (direction + magnitude), shaft width, colour
Helix	`helix()`	Position, axis, radius, coils, colour (spring-like)
Label	`label()`	Position, text, size, colour — displays text in 3D
Local Light	`local_light()`	Adds a point light source at a given position
Ground plane (preset)	`box()` flat	Pre-configured floor geometry

Motion Category

BLOCK	DESCRIPTION
Set Velocity	Sets `object.velocity` to a vector value
Apply Force	Computes acceleration from force and mass; updates velocity
Update Velocity	`velocity += acceleration * dt`
Update Position	`pos += velocity * dt`
Bounce (ground)	Reflects the y-component of velocity with a restitution coefficient
Set Property	Sets any named property on an object (e.g., `ball.color = color.blue`)
Increment Property	Adds a value to an object property

State Category

BLOCK	DESCRIPTION
Define Constant	Creates a named constant (`NAME = VALUE`) at the top level
Set Variable	`variable = expression`
Telemetry Display	Shows a live text label with multiple variable values

Control Category

BLOCK	DESCRIPTION
Forever Loop	`while True:` with `rate(fps)`
For Range Loop	`for i in range(start, stop, step):`
Rate	`rate(fps)` — limits loop speed
Time Step (`dt`)	Sets the `dt` time increment variable
If	Conditional: runs blocks if condition is true
If-Else	Two-branch conditional
Break	Exits the current loop
Comment	Non-executing text annotation

Values Category

BLOCK	DESCRIPTION
Vector	Creates `vector(x, y, z)`
Colour	Colour picker — named colour or custom hex; outputs `color.xxx` or `vector(r,g,b)`
Physics Constant	Dropdown of common constants: `g` , `G` , `c` , `h` , `e` , `me` , `mp` , `k_e` , `ε₀` , `μ₀` , `σ` , `R` , `NA`
Variable Read	Reads a named variable
Expression	Raw Python expression — use for any value not covered by other blocks

3D Math Category

BLOCK	DESCRIPTION
Magnitude	`mag(vector)`
Norm	`norm(vector)` — unit vector
Cross product	`cross(a, b)`
Dot product	`dot(a, b)`
Scale vector	`scalar * vector`
Add vectors	`vector_a + vector_b`
Get component	`.x`, `.y`, `.z` of a vector
Power	`a ** b`
Clamp	`clamp(val, lo, hi)`
Rotate object	Rotates an object by angle around an axis vector
Scene / Camera	Sets `scene.center`, `scene.range`, `scene.forward`, `scene.up`

15. VPython Quick Reference

Creating Objects

```
# Sphere
ball = sphere(pos=vector(0,5,0), radius=1, color=color.red)

# Box
floor = box(pos=vector(0,-1,0), size=vector(20,0.2,20), color=color.green)

# Cylinder
rod = cylinder(pos=vector(0,0,0), axis=vector(0,5,0), radius=0.2, color=color.white)

# Arrow (shows direction/magnitude)
v_arrow = arrow(pos=ball.pos, axis=ball.velocity*0.1, shaftwidth=0.1, color=color.yellow)

# Helix (spring)
spring = helix(pos=vector(-5,0,0), axis=vector(10,0,0), radius=0.5, coils=12)

# Label
info = label(pos=vector(0,8,0), text="Hello!", height=14, box=False)
```

Physics Pattern (Newton's 2nd Law)

```
GlowScript 3.2 VPython

# Setup
ball = sphere(pos=vector(0,10,0), radius=0.5, color=color.cyan, make_trail=True)
ball.mass = 2.0
ball.velocity = vector(3, 0, 0)

g    = vector(0, -9.81, 0)
drag = 0.1
dt   = 0.005

while True:
    rate(200)
    F_net = ball.mass * g - drag * ball.velocity
    ball.velocity += (F_net / ball.mass) * dt
    ball.pos      += ball.velocity * dt
```

Vectors

```
v = vector(1, 2, 3)      # create
m = mag(v)               # magnitude = sqrt(14)
u = norm(v)              # unit vector
d = dot(v, v)            # dot product = 14
c = cross(v, vector(1,0,0))  # cross product
```

Colors

```
color.red; color.green; color.blue; color.white; color.black
color.yellow; color.cyan; color.magenta; color.orange
vector(0.8, 0.2, 0.5)  # custom: (R, G, B) in [0,1]
color.gray(0.5)          # mid-grey
```

Scene configuration

```
scene.title    = "My Simulation"
scene.range    = 10          # half-width of view
scene.center   = vector(0,0,0)
scene.forward  = vector(0,-0.3,-1)
scene.up       = vector(0,1,0)
scene.background = vector(0.05, 0.08, 0.15)
scene.ambient  = color.gray(0.3)
```

---

16. Building Your First Simulation (Tutorial)

Goal: Falling ball bounces on a floor

This tutorial walks through building a simple elastic bounce simulation using blocks.

Step 1 — Start a blank blocks project

From the Start Menu, click **Blank (Blocks)**. You will see an empty block canvas.

Step 2 — Add Simulation Start

From the **Starter** toolbox category, drag a **Simulation Start** block onto the canvas.

- Set the title to `"Bouncing Ball"`.

Step 3 — Create a sphere

From **Objects**, add a **Quick Sphere** block inside the Simulation Start.

- Set `NAME = ball`, position `Y = 5`, radius `1`, colour: red.

Step 4 — Create a floor

From **Objects**, add a **Quick Box** block.

- Set `NAME = floor`, position `Y = -0.5`, size `W = 20, H = 1, D = 10`, colour: green.

Step 5 — Set gravity

From **Starter**, drag a **Set Gravity** block. It defaults to `g = vector(0, -9.81, 0)`.

Step 6 — Set time step

Drag a **Time Step** block. Leave `dt = 0.01`.

Step 7 — Set initial velocity

From **Motion**, drag a **Set Velocity** block. Set it to `ball`, velocity `vector(2, 0, 0)`.

Step 8 — Add the main loop

From **Control**, drag a **Forever Loop**. Inside it:

1. **Rate** block: `60` fps.
2. **Update Position** block: `ball`, `dt`.

Step 9 — Add bounce logic

Still inside the Forever Loop, add an **If** block from Control.

- Condition: set a **Compare** from Logic: `ball.pos.y < ball.radius`
- Inside the If: add **Bounce (ground)** from Motion, set `ball`, restitution `0.8`.

Step 10 — Run!

Click **Run**. The red ball should fall, bounce, and drift to the right.

Experiment:

- Change the restitution value to `1.0` (perfectly elastic) or `0.3` (damped).
- Add drag: modify velocity by `velocity * 0.998` each loop iteration.
- Add a second ball at a different starting position.

---

17. For Educators

Classroom use

Physics IDE is designed to be used in:

- **Introductory physics labs** — students verify equations with simulations.
- **Computational physics courses** — scaffold from blocks to full Python coding.
- **Engineering design exercises** — students iterate on parameters to meet a physical objective.

Setting up a template assignment

1. Build (or open) a simulation with some parameters left intentionally wrong or incomplete.
2. Click **Export .xml** to download the block workspace.
3. Distribute the `.xml` file to students.
4. Students click **Import** to load the workspace and complete the assignment.

Assessing student work

Students can:

- Export their finished simulation as `.py` and submit it.
- Export a **blocks PDF** showing their block structure.
- Export a **code PDF** with syntax highlighting.
- Download a **CSV from the Trace Table** showing recorded variable data over time.

iKamva (Sakai) integration

Physics IDE can be embedded directly into an iKamva course site using the **Web Content** tool. Ask your ICT/e-learning support team to:

1. Enable the Web Content tool in your course.
2. Add a new Web Content item with the Physics IDE URL.

Students will see the IDE inside iKamva without leaving the LMS. See the companion **Technical Architecture document** for full configuration details.

Beginner Mode tips

Toggle **Beginner Mode** from the toolbar to reduce the block palette to only essential categories. This is highly recommended for:

- First-year students and school-level learners.
- Early lab sessions where cognitive load should be minimal.

Block → Code progression

A natural classroom progression:

1. **Week 1–2**: Use precoded examples; press Run; observe what changes when values change.
2. **Week 3–4**: Build from starter blocks in Beginner Mode.
3. **Week 5–6**: Unlock full Advanced toolbox; add custom `python_raw_block` snippets.
4. **Week 7+**: Switch to Code mode; write VPython directly.
